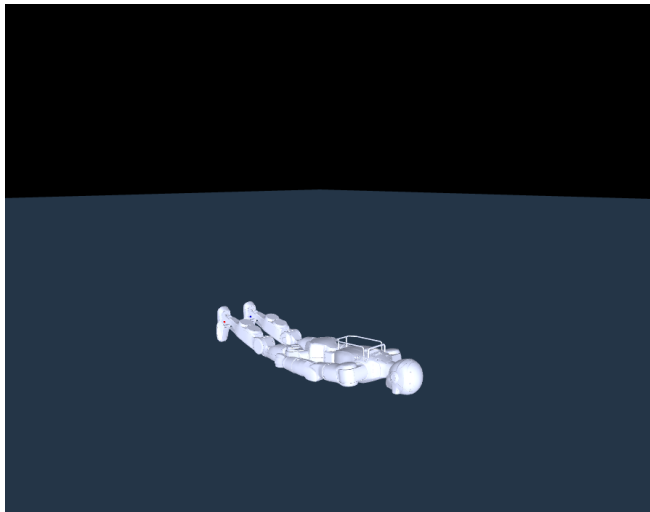


Embodied AI

Lab Manual for MuJoCo

Lab 3 — Where It Breaks

September 2026



Shaoqin Li

Applied and Embodied AI Lab
Electrical Engineering Department
The City College of New York

you build the second robot yourself · no GPU · nothing new to install

Contents

The shape of this lab	4
What you will do	4
What you should be able to say afterwards	4
Part 0 — Build the <code>exp3</code> workspace	5
The two things you download	5
Part 1 — Build the second robot.....	6
Part 2 — Meet both policies	10
Part 3 — Create the measuring tool	11
Part 4 — Attack 1: delay	16
Predict first.....	16
Run it.....	17
Watch the row	17
Why — three reasons, and you should be able to give all three	18
Part 5 — Attack 2: corrupt one sensor	18
Predict first.....	18
The null control comes first	18
The hunt.....	19
Why gravity	19
The two channels that are not sensors.....	20
Part 6 — Attack 3: swap the simulator	20
Predict first.....	20
Run it.....	20
Read it before you move on	21
Watch them peel apart	21
The control — turn the feedback off	26
An honest limitation, before anyone asks	27
Part 7 — Synthesis	27
Change something	27
What this lab was really teaching	28
Troubleshooting.....	28

What to hand in	28
Appendix — Instructor Notes	29

Hardware: any laptop, no GPU · **Before you start:** finish Labs 1 and 2. **Nothing new to install.** This lab needs exactly what Lab 1 needed — `mujoco` and `numpy`. No PyTorch, no stable-baselines3, no training. You build the second robot yourself out of the file Unitree publish, from the clone you already made in Lab 1.

Copy the code from the `.md` version of this manual, not from the PDF. Open `LAB3_where_it_breaks.md` in VS Code beside this document and copy from there.

Every **Expected** block below was produced by actually running the command. **The policies are frozen and the physics has no randomness**, so unlike Lab 2 your numbers should match these to the last digit. **If they differ, that is a finding — tell your instructor.** Verified 2026-09-01 on Ubuntu 24.04 under WSL2, MuJoCo 3.12.0, NumPy 2.4.6.

The shape of this lab

Labs 1 and 2 asked *can it stand?* Both answers were yes. This lab asks the question that decides whether any of it would work on real hardware: **what breaks it, and which of those breakages would happen on a real robot?**

You attack two finished policies three ways. Nothing trains. Nothing is random. You are measuring.

What you will do

Part		What you end up with
0	Check what Labs 1 and 2 left you	a workspace and two policies
1	Build the second robot	<code>model/r1_walk_deploy_env.xml</code> , from Unitree's own file
2	Meet both policies	the baseline everything is measured against
3	Create the measuring tool	<code>exp3_sweep.py</code>
4	Attack 1: delay — make the controller late	a 6× difference
5	Attack 2: corrupt one sensor — find the one that matters	a null control, then a result
6	Attack 3: swap the simulator — and watch nothing fall	the finding of the lab
7	Synthesis — where the sim2sim gap actually lives	

What you should be able to say afterwards

1. Why a walking policy tolerates far less control delay than a standing one — three reasons.
2. Which single sensor a balance policy cannot survive losing, and why that one.
3. **Why the sim2sim gap is a drift and not a fall** — and what keeps it bounded.
4. What a null control is, and why a result without one is not evidence.

Part 0 — Build the **exp3** workspace

Same pattern as Lab 2: a folder of its own, borrowing what you have already built.

```
mkdir -p ~/r1_lab/exp3/model ~/r1_lab/exp3/policies
cd ~/r1_lab/exp3
cp -r ~/r1_lab/exp1/model/assets model/assets
cp ~/r1_lab/exp2/model/r1_walk_train.xml model/
cp ~/r1_lab/exp2/watch_policy.py ~/r1_lab/exp2/render_policy.py .
```

That is Lab 1's meshes, Lab 2's robot, and Lab 2's viewer. **Every command in this lab runs from `~/r1_lab/exp3`.**

The two things you download

Same reason as Lab 2, twice over: a trained policy cannot be typed. Your instructor gives you **lab3_policies.tar.gz** (640 KB).

```
tar -xzf ~/Downloads/lab3_policies.tar.gz -C ~/r1_lab/exp3
ls policies
```

Expected:

```
exp2_stand.npz  walk3b.npz
```

exp2_stand.npz is the standing policy you evaluated in Lab 2. **walk3b.npz** is new — a *walking* policy, trained for this project over four warm-started runs totalling 3.7M steps. You will not train one; walking does not train cleanly from scratch in a lab session, and that fact is itself a result you meet in Part 4.

```
python -c "import mujoco, numpy; print(mujoco.__version__, numpy.__version__)"
```

Expected:

```
3.12.0 2.4.6
```

☒ **Checkpoint: 43 meshes, one robot, two policies, no new installs.**

Part 1 — Build the second robot

Everything you have measured so far happened inside **one** description of the R1 — the one you wrote in Lab 1 and re-actuated in Lab 2. A policy that works there is not guaranteed to work anywhere else, and that gap is what this lab exists to measure. So you need a second robot.

You already have it. Unitree publish `R1_C++.xml`, the description their **own simulator** loads, and you cloned it in Lab 1. It is the same machine, described by different people for a different purpose: different collision shapes, different joint damping, different actuators.

Rather than hand you the finished file, here is the script that makes it, so that nothing about the second robot is mysterious. Create `build_deploy_model.py`:

```
"""Build the SECOND simulator's robot from Unitree's own file -- Lab 3.

Lab 2 trained against model/r1_walk_train.xml, which you made from Lab 1's
robot by swapping its actuators. Unitree ship a different description of the
same machine -- R1_C++.xml, the one their own simulator loads -- and a policy
that works in one is not guaranteed to work in the other. That gap is what
Lab 3 measures.

This reads Unitree's file straight out of the clone you made in Lab 1 and
applies six changes, printing each one, so nothing about the second model is
mysterious:

1. meshdir -> the assets folder beside this script
2. remove five phantom bodies Unitree park at z = 20 (wrists, waist pitch)
3. add a floor (there is none: their scene.xml supplies it)
4. name the two ankle foot boxes, so the environment can find foot contacts
5. replace their 29 <motor> actuators with our 24 <position> actuators, at the
   deployment PD gains AND the deployment torque limits
6. add the two IMU sensors the policy's observation is built from

python build_deploy_model.py
"""
import argparse
import os
import xml.etree.ElementTree as ET

HERE = os.path.dirname(os.path.abspath(__file__))
DEFAULT_SRC = os.path.join(HERE, os.pardir, "unitree_mujoco",
                           "unitree_robots", "r1", "R1_C++.xml")

PHANTOM_BODIES = ["waist_pitch", "left_wrist_pitch", "left_wrist_yaw",
                  "right_wrist_pitch", "right_wrist_yaw"]

# (actuator name, joint, kp, kv) -- the gains the deployment controller applies
ACTS = [
    ("left_hip_pitch", "left_hip_pitch_joint", 150.0, 8.0),
    ("left_hip_roll", "left_hip_roll_joint", 150.0, 8.0),
    ("left_hip_yaw", "left_hip_yaw_joint", 150.0, 8.0),
    ("left_knee", "left_knee_joint", 200.0, 10.0),
    ("left_ankle_pitch", "left_ankle_pitch_joint", 60.0, 4.0),
```

```

("left_ankle_roll", "left_ankle_roll_joint", 60.0, 4.0),
("right_hip_pitch", "right_hip_pitch_joint", 150.0, 8.0),
("right_hip_roll", "right_hip_roll_joint", 150.0, 8.0),
("right_hip_yaw", "right_hip_yaw_joint", 150.0, 8.0),
("right_knee", "right_knee_joint", 200.0, 10.0),
("right_ankle_pitch", "right_ankle_pitch_joint", 60.0, 4.0),
("right_ankle_roll", "right_ankle_roll_joint", 60.0, 4.0),
("waist_roll", "waist_roll_joint", 80.0, 5.0),
("waist_yaw", "waist_yaw_joint", 80.0, 5.0),
("left_shoulder_pitch", "left_shoulder_pitch_joint", 60.0, 4.0),
("left_shoulder_roll", "left_shoulder_roll_joint", 60.0, 4.0),
("left_shoulder_yaw", "left_shoulder_yaw_joint", 40.0, 3.0),
("left_elbow", "left_elbow_joint", 40.0, 3.0),
("left_wrist_roll", "left_wrist_roll_joint", 40.0, 3.0),
("right_shoulder_pitch", "right_shoulder_pitch_joint", 60.0, 4.0),
("right_shoulder_roll", "right_shoulder_roll_joint", 60.0, 4.0),
("right_shoulder_yaw", "right_shoulder_yaw_joint", 40.0, 3.0),
("right_elbow", "right_elbow_joint", 40.0, 3.0),
("right_wrist_roll", "right_wrist_roll_joint", 40.0, 3.0),
]

# deployment motor torque limits (their <motor ctrlrangle>). A <position>
# actuator is otherwise UNLIMITED -- this is the key missing constraint: the
# deployment PD saturates at these, the training one never does.
TORQUE_LIM = {"hip": 88.0, "knee": 139.0, "ankle": 50.0,
              "waist_roll": 50.0, "waist_yaw": 88.0, "arm": 25.0}

def force_limit(joint):
    if "hip" in joint:
        return TORQUE_LIM["hip"]
    if "knee" in joint:
        return TORQUE_LIM["knee"]
    if "ankle" in joint:
        return TORQUE_LIM["ankle"]
    if joint == "waist_roll_joint":
        return TORQUE_LIM["waist_roll"]
    if joint == "waist_yaw_joint":
        return TORQUE_LIM["waist_yaw"]
    return TORQUE_LIM["arm"] # shoulders, elbow, wrist_roll

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--src", default=DEFAULT_SRC, help="Unitree's R1_C++.xml")
    ap.add_argument("--out", default=os.path.join(HERE, "model",
                                                  "r1_walk_deploy_env.xml"))

    a = ap.parse_args()

    src = os.path.normpath(a.src)
    if not os.path.exists(src):
        raise SystemExit("cannot find %s -- is your Lab 1 clone still there?" % src)
    print("reading", src)
    tree = ET.parse(src)

```

```

root = tree.getroot()
parent = {c: p for p in root.iter() for c in p}

# 1. meshes live beside the file we are writing
root.find("compiler").set("meshdir", "assets")
print(" 1. meshdir -> assets")

# 2. five bodies parked at z = 20 with nothing attached to them
world = root.find("worldbody")
for b in list(world):
    if b.tag == "body" and b.get("name") in PHANTOM_BODIES:
        world.remove(b)
        print(" 2. removed phantom body", b.get("name"))

# 3. their scene.xml has the floor; this file has none
ET.SubElement(world, "geom", {"name": "floor", "type": "plane",
                              "size": "20 20 0.1", "friction": "1 1 1",
                              "rgba": "0.3 0.3 0.35 1"})
print(" 3. added floor plane, friction 1 1 1")

# 4. the environment finds foot contacts by geom NAME; theirs are unnamed
for g in root.iter("geom"):
    if g.get("size") == "0.09 0.025 0.0075" and g.get("type") == "box":
        body = parent[g]
        side = "left" if "left" in (body.get("name") or "") else "right"
        g.set("name", "%s_foot_collision" % side)
        print(" 4. named foot box:", g.get("name"))

# 5. their 29 <motor> actuators -> our 24 <position> ones
old = root.find("actuator")
n_old = len(list(old)) if old is not None else 0
if old is not None:
    root.remove(old)
act = ET.SubElement(root, "actuator")
for name, joint, kp, kv in ACTS:
    fl = force_limit(joint)
    ET.SubElement(act, "position",
                  {"name": name, "joint": joint, "kp": "%g" % kp, "kv": "%g" % kv,
                   "forcerange": "%g %g" % (-fl, fl), "forcelimited": "true"})
print(" 5. replaced %d <motor> actuators with %d <position> actuators"
      % (n_old, len(ACTS)))

# 6. the observation is built from these two
sensor = root.find("sensor")
if sensor is None:
    sensor = ET.SubElement(root, "sensor")
phantom_joints = {b + "_joint" for b in PHANTOM_BODIES}
for s in list(sensor):
    if s.get("joint") in phantom_joints:
        sensor.remove(s)
have = {s.get("name") for s in sensor}
for name, tag in (("imu_ang_vel", "gyro"), ("imu_lin_vel", "velocimeter")):
    if name not in have:
        ET.SubElement(sensor, tag, {"name": name, "site": "imu"})

```



```

        print(" 6. added sensor", name)

    os.makedirs(os.path.dirname(a.out), exist_ok=True)
    tree.write(a.out, encoding="unicode")
    print("wrote", a.out)

    import mujoco
    m = mujoco.MjModel.from_xml_path(a.out)
    print("compiled: bodies %d joints %d actuators %d mass %.2f kg"
          % (m.nbody, m.njnt, m.nu, sum(m.body_mass)))
    assert m.nu == 24, m.nu
    for s in ("imu_ang_vel", "imu_lin_vel"):
        assert mujoco.mj_name2id(m, mujoco.mjtObj.mjOBJ_SENSOR, s) >= 0, s
    print("OK")

if __name__ == "__main__":
    main()

```

```
python build_deploy_model.py
```

Expected:

```

reading /home/YOURNAME/r1_lab/unitree_mujoco/unitree_robots/r1/R1_C++.xml
1. meshdir -> assets
2. removed phantom body waist_pitch
2. removed phantom body left_wrist_pitch
2. removed phantom body left_wrist_yaw
2. removed phantom body right_wrist_pitch
2. removed phantom body right_wrist_yaw
3. added floor plane, friction 1 1 1
4. named foot box: left_foot_collision
4. named foot box: right_foot_collision
5. replaced 29 <motor> actuators with 24 <position> actuators
6. added sensor imu_ang_vel
6. added sensor imu_lin_vel
wrote /home/YOURNAME/r1_lab/exp3/model/r1_walk_deploy_env.xml
compiled: bodies 26 joints 25 actuators 24 mass 28.93 kg
OK

```

28.93 kg, 26 bodies, 25 joints — the same numbers you printed in Lab 1 and Lab 2. It is the same robot. Six changes were needed to make Unitree's file *runnable* for our purpose, and none of them touch a mass, a link length or a joint limit.

Read the six lines again, because the honesty of the whole lab rests on them:

#	Change	Why it is not cheating
1	meshdir	where the shapes live on <i>your</i> disk

#	Change	Why it is not cheating
2	five phantom bodies removed	they sit at $z = 20$ m with nothing attached
3	floor added	their <code>scene.xml</code> supplies it; this file has none
4	foot boxes named	our environment finds foot contacts by name
5	actuators replaced	so both robots are driven the same way — otherwise you would be measuring a different controller, not a different world
6	two IMU sensors	the policy's observation is built from them

What is left different is the physics: the collision geometry (their full meshes and foot boxes versus our capsules), the joint damping and friction, and the deployment torque limits. That difference is the entire subject of Part 6.

☒ Checkpoint: **OK**, 24 actuators, 28.93 kg.

Part 2 — Meet both policies

Establish the baseline before attacking anything. First the standing policy from Lab 2:

```
python watch_policy.py --headless
```

Expected:

```
exp2_stand.npz
STOOD the full 12s (max tilt 3.35 deg)
```

Now the one you have not seen — the walking policy, asked to move forward at 0.5 m/s:

```
python watch_policy.py --policy walk3b.npz --vx 0.5 --headless
```

Expected:

```
walk3b.npz | vx 0.5
STOOD the full 12s (max tilt 10.94 deg)
```

Both work. Note the tilt figures — **3.35° standing, 10.94° walking**. The walker is leaning three times as far, all the time, because that is what walking *is*: falling forward and catching yourself, over and over.

Hold on to that. Standing stabilises a **fixed point** — one pose, error decays toward it. Walking stabilises a **limit cycle** — a repeating orbit, never at rest. Every result in this lab comes back to that difference.

If your machine opens a MuJoCo window, drop `--headless` from either command and watch. If you get `gladLoadGL error` — the WSL2 problem from Lab 1 Part 9b — use `render_policy.py` for pictures.

☒ **Checkpoint: both policies work before you attack them. A broken baseline invalidates everything after it.**

Part 3 — Create the measuring tool

One script runs all three attacks. Create `exp3_sweep.py`:

```
"""Experiment 3 -- Where it breaks. The measuring tool for Lab 3.

Three attacks on two FROZEN policies. No training, no PyTorch, no randomness in
the physics: every number here reproduces exactly on your machine.

    python exp3_sweep.py latency      # how much control delay each policy survives
    python exp3_sweep.py noise       # which sensor the standing policy needs
    python exp3_sweep.py sim2sim     # what happens when you swap the simulator
    python exp3_sweep.py all         # all three, ~3 minutes

Every table this prints has a matching command in watch_policy.py, so you can
WATCH any row you find interesting. The tool tells you where to look; the
viewer shows you what it looks like.

KEYS TO READING THE OUTPUT
    survived    did it stay upright for the whole episode
    fell at     when it hit 45 degrees of tilt
    -----    the row where a policy crosses from working to broken
"""
import argparse, os, sys, time
import numpy as np
import mujoco

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
from watch_policy import Robot, load_policy, CH, projected_gravity, LEG # noqa: E402

# ----- EDIT ZONE 1
# Which policies get attacked. (file, forward command, label)
POLICIES = [("exp2_stand.npz", 0.0, "standing"),
            ("walk3b.npz", 0.5, "walking")]

# EDIT ZONE 2 -- control delay in 20 ms steps. 4 = 80 ms.
DELAYS = [0, 1, 2, 3, 4, 6, 8, 12]
```

```

# EDIT ZONE 3 -- sensor noise doses, in multiples of that channel's TRAINING noise
DOSES = [1, 5, 10, 20]
SENSORS = ["angvel", "gravity", "dofpos", "dofvel"] # the four with training noise
SEEDS = [0, 1, 2]

# EDIT ZONE 4 -- how long each episode runs, and when sim2sim reports
SECONDS = 10.0
# sim2sim runs longer on purpose: the walking gap is an INTEGRATION effect and
# needs time on the clock before it is visible
SIM2SIM_SECONDS = 20.0
REPORT_AT = [0.5, 1.0, 2.0, 5.0, 10.0, 20.0]
# -----

FELL = 45.0 # degrees of tilt that counts as fallen

def rollout(policy, vx, seconds=SECONDS, latency=0, noise=None, mult=0.0,
            zero=None, seed=0, xml=None, abs_sigma=None):
    """Run one episode. Returns (survived_seconds, hit_the_cap, max_tilt)."""
    xml = xml or os.path.join(HERE, "model", "r1_walk_train.xml")
    path = os.path.join(HERE, "policies", policy)
    act, stats = load_policy(path), np.load(path)
    r, rng = Robot(xml, vx), np.random.default_rng(seed)

    sl = sigma = None
    if noise:
        sl, sig, scale = CH[noise]
        n = sl.stop - sl.start
        if abs_sigma is not None:
            sigma = np.full(n, abs_sigma) # dose in raw units, not multiples
        elif sig is not None:
            sigma = np.full(n, sig * scale) # multiples of TRAINING noise
        else:
            sigma = np.sqrt(stats["obs_var"][sl]) # multiples of its own spread
    zsl = CH[zero][0] if zero else None
    zval = stats["obs_mean"][zsl] if zero else None

    peak, total = 0.0, int(seconds / r.dt)
    for s in range(total):
        r.buf.append(act(r.obs(sl, sigma, mult, rng, zsl, zval)))
        r.step(r.buf[max(0, len(r.buf) - 1 - latency)])
        peak = max(peak, r.tilt())
        if r.tilt() > FELL:
            return s * r.dt, False, peak
    return seconds, True, peak

def fmt(sec, ok, seconds=SECONDS):
    return f"{'survived':>8s}" if ok else f"{sec:6.2f}s "

# ===== ATTACK 1: DELAY
def sweep_latency():

```

```

print("\n" + "=" * 62)
print("ATTACK 1 -- CONTROL DELAY")
print("A real robot cannot act instantly: sensor -> network -> compute ->")
print("motor all cost time. --latency N holds each action for N x 20 ms.")
print("=" * 62)
head = " delay " + "".join(f"{lab:>14s}" for _, _, lab in POLICIES)
print(head + "\n " + "-" * (len(head) - 2))
# the margin is the last delay that survived with NO failure below it --
# not simply the last row that happened to survive
margin = {lab: 0 for _, _, lab in POLICIES}
alive = {lab: True for _, _, lab in POLICIES}
for n in DELAYS:
    row = f" {n*20:4d} ms "
    for pol, vx, lab in POLICIES:
        sec, ok, _ = rollout(pol, vx, latency=n)
        row += f"{fmt(sec, ok):>14s}"
        if ok and alive[lab]:
            margin[lab] = n * 20
        elif not ok:
            alive[lab] = False
    print(row)
print()
for lab, ms in margin.items():
    print(f" {lab:9s} tolerates {ms} ms with nothing failing below it")
a, b = list(margin.values())
if a and b:
    print(f"\n >> The standing policy's delay budget is {a/b:.0f}x the walking policy's.")
print("""
NOTE ON THIS NUMBER. Every row above is ONE deterministic rollout, which is
why it reproduces exactly for you. The research run used 20 randomised
episodes over 20 s and reported the fully-safe margins as 80 ms standing vs
20 ms walking -- a 4x gap. A single clean rollout is more forgiving than 20
disturbed ones, so your ratio here will look bigger. The CLIFF LOCATION is
what reproduces; the exact survival second is not. Raise SEEDS and add a
random push if you want the harsher number.""")
print("""
WHY, three reasons -- be able to say all three:
1. SUPPORT POLYGON. Standing keeps both feet down. Walking spends part of
every cycle on one foot, with a window where no corrective ankle torque
exists at all.
2. TIME CONSTANT. Balance is an inverted pendulum, tau = sqrt(L/g) ~ 0.25 s
at this CoM height. 40 ms is 16% of the fall time -- survivable standing.
3. FIXED POINT vs LIMIT CYCLE. Standing stabilises a point and error decays.
Walking stabilises a repeating orbit, so delay shifts phase and the phase
error COMPOUNDS every step instead of dying out.

WATCH IT: python watch_policy.py --policy walk3b.npz --vx 0.5 --latency 2""")

# ===== ATTACK 2: SENSORS
def sweep_noise():
    pol, vx, lab = POLICIES[0]
    print("\n" + "=" * 62)
    print("ATTACK 2 -- CORRUPT ONE SENSOR AT A TIME")

```

```

print("The observation is 45 numbers in six groups. We add noise to ONE")
print("group, dosed in multiples of the noise that group already saw in")
print("TRAINING -- so x10 means ten times what the policy expects.")
print("=" * 62)

print("\n FIRST, THE NULL CONTROL.")
print(" The standing policy was trained with a fixed command and has never")
print(" seen a different one. Pinning that channel to its training value")
print(" must change NOTHING. If it does, the tool is broken and no result")
print(" below can be trusted.")
base_s, base_ok, base_t = rollout(pol, vx)
null_s, null_ok, null_t = rollout(pol, vx, zero="command")
print(f"\n      clean          max tilt {base_t:6.2f} deg   {fmt(base_s, base_ok)}")
print(f"      command zeroed  max tilt {null_t:6.2f} deg   {fmt(null_s, null_ok)}")
ok = abs(base_t - null_t) < 1e-6 and base_ok == null_ok
print(f"      -> {'PASS, identical. The tool is honest.' if ok else 'FAIL -- STOP AND INVESTIGATE.'}")

print(f"\n NOW HUNT. {lab} policy, {len(SEEDS)} seeds per cell.")
print(" These four are the real sensors. All are dosed in multiples of")
print(" their OWN training noise, so the columns compare fairly.\n")
print(" sensor          " + "".join(f"{'x'+str(d):>10s}" for d in DOSES))
print(" " + "-" * (15 + 10 * len(DOSES)))
breaks = {}
for ch in SENSORS:
    row, broke = f" {ch:<15s}", None
    for d in DOSES:
        n = sum(rollout(pol, vx, noise=ch, mult=d, seed=s)[1] for s in SEEDS)
        row += f"{'{n}/{len(SEEDS)}':>10s}"
        if n == 0 and broke is None:
            broke = d
    breaks[ch] = broke
print(row + (" if broke is None else f" <- breaks at x{broke}"))

ranked = sorted([c for c in SENSORS if breaks[c]], key=lambda c: breaks[c])
first = ranked[0] if ranked else None
held = [c for c in SENSORS if breaks[c] is None]
print(f"""
>> {first} is the first sensor to break, at x{breaks[first]}."" if first else ""
print(f" >> {' , '.join(held)} survive every dose tested, up to x{DOSES[-1]}."
print(f""

WHY: projected gravity is the ONLY absolute orientation reference in the
whole observation. Joint sensors are all relative to the body. Angular
velocity is a rate that has to be integrated and drifts. Corrupt gravity and
the policy loses which way is up -- unrecoverable for a balance task.

A PREDICTION THAT WAS WRONG: the pre-registration said angvel would be
severe. It is among the most robust. Written down beforehand, so it counts.

THE TWO CHANNELS THAT ARE NOT SENSORS. command and prev_action had no
training noise, so "multiples of training noise" is undefined for them and
they do NOT belong in the table above. command is the null control you ran
first. prev_action is the policy's own last output fed back to it, so we
dose it in raw units instead:

```

```

"""
    for sd in [0.1, 0.25, 0.5, 1.0]:
        n = sum(rollout(pol, vx, noise="prev_action", abs_sigma=sd, mult=1.0,
                        seed=s)[1] for s in SEEDS)
        print(f"    prev_action noise sd {sd:<5.2f}    {n}/{len(SEEDS)} survived")
    print("""
Its own values run about +/-1, so sd 0.5 is already heavy corruption.

WATCH IT:  python watch_policy.py --noise gravity 10""")

# ===== ATTACK 3: SIM2SIM
def yaw(q):
    w, x, y, z = q
    return np.degrees(np.arctan2(2*(w*z + x*y), 1 - 2*(y*y + z*z)))

def sweep_sim2sim():
    print("\n" + "=" * 62)
    print("ATTACK 3 -- SAME POLICY, DIFFERENT SIMULATOR")
    print("Identical frozen weights, same kinematic tree, different collision")
    print("geometry. Both robots run their own copy of the policy and react to")
    print("their own observations. How far apart do they drift?")
    print("=" * 62)
    A = os.path.join(HERE, "model", "r1_walk_train.xml")
    B = os.path.join(HERE, "model", "r1_walk_deploy_env.xml")

    for pol, vx, lab in POLICIES:
        path = os.path.join(HERE, "policies", pol)
        act = load_policy(path)
        ra, rb = Robot(A, vx), Robot(B, vx)
        print(f"\n {lab.upper()} ({pol})")
        print("    t    leg RMS    base apart    heading apart    upright")
        print("    " + "-" * 58)
        marks = [t for t in REPORT_AT if t <= SIM2SIM_SECONDS]
        total = int(SIM2SIM_SECONDS / ra.dt)
        for s in range(total):
            for r in (ra, rb):
                a = act(r.obs())
                r.buf.append(a)
                r.step(a)
            t = (s + 1) * ra.dt
            if marks and t >= marks[0] - 1e-9:
                marks.pop(0)
                rms = np.sqrt(np.mean((ra.d.qpos[ra.qadr] - rb.d.qpos[rb.qadr]) ** 2))
                dist = np.linalg.norm(ra.d.qpos[:2] - rb.d.qpos[:2])
                dh = abs(yaw(ra.d.qpos[3:7]) - yaw(rb.d.qpos[3:7]))
                up = "both" if max(ra.tilt(), rb.tilt()) < FELL else "FALLEN"
                print(f"    {t:5.1f}s    {rms:7.4f} rad    {dist:7.3f} m    "
                      f"{dh:7.1f} deg    {up}")
        print("""
>> NEITHER ONE FALLS. The sim2sim gap is not a fall.
>> Standing clamps joint divergence near 0.01 rad and stays put.
>> Walking clamps near 0.10 rad -- and because walking stabilises a LIMIT

```

CYCLE, that bounded per-step difference in step geometry INTEGRATES. A thousand control steps later the two simulations are metres apart and facing different directions, with both robots still upright.

Turn the feedback off and the picture changes completely: replay one sim's actions blindly into the other and divergence doubles every 0.54-0.68 s and topples the robot in ~2.5 s. Feedback BOUNDS the physics gap. What differs between the two policies is the level it clamps at -- about 10x.

LIMITATION, say it before you are asked: without hardware neither simulator is ground truth. This measures DISAGREEMENT, not error.

WATCH IT: `python watch_sim2sim.py --policy walk3b.npz --vx 0.5""")`

```
if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("attack", choices=["latency", "noise", "sim2sim", "all"])
    a = p.parse_args()
    t0 = time.time()
    if a.attack in ("latency", "all"): sweep_latency()
    if a.attack in ("noise", "all"): sweep_noise()
    if a.attack in ("sim2sim", "all"): sweep_sim2sim()
    print(f"\n ({time.time() - t0:.0f} s)\n")
```

It has three **EDIT ZONE** comments in it. You are meant to change those numbers in Part 7 — the tool is not a black box, and every table it prints comes from a list you can see.

Tool	What it is for
<code>exp3_sweep.py</code>	the measuring tool — runs the attacks, prints the tables
<code>watch_policy.py</code>	watches one policy under one attack — see any row you find interesting
<code>watch_sim2sim.py</code>	runs the same policy in two simulators at once (Part 6)

The sweep tells you where to look. The viewers show you what it looks like. Every table row has a matching **WATCH IT:** line printed underneath it.

Part 4 — Attack 1: delay

A real robot cannot act instantly. Sensor → network → compute → motor all cost time, and the action that finally reaches the joint was computed for a robot that has since moved. `--latency N` holds each action for $N \times 20$ ms.

Predict first

Which policy do you think survives more delay, and by how much? **Write it down.**

Run it

```
python exp3_sweep.py latency
```

About 16 seconds. Expected:

delay	standing	walking
0 ms	survived	survived
20 ms	survived	survived
40 ms	survived	4.26s
60 ms	survived	1.56s
80 ms	survived	1.80s
120 ms	survived	1.80s
160 ms	2.44s	1.94s
240 ms	1.54s	1.18s

standing tolerates 120 ms with nothing failing below it
walking tolerates 20 ms with nothing failing below it

Standing's delay budget is six times the walking policy's. Same robot, same simulator, same kind of network — the only difference is what the policy is stabilising.

Watch the row

```
python watch_policy.py --policy walk3b.npz --vx 0.5 --latency 2 --headless
```

Expected:

```
walk3b.npz | vx 0.5 | latency 40 ms  
fell at 4.26s
```

```
python watch_policy.py --latency 4 --headless
```

Expected:

```
exp2_stand.npz | latency 80 ms  
STOOD the full 12s (max tilt 5.32 deg)
```

Double the delay, and the stander is still up — merely leaning further than its clean 3.35°.

Why — three reasons, and you should be able to give all three

1. **Support polygon.** Standing keeps both feet down, so an ankle can always push back. Walking spends part of every cycle on one foot, with a window where no corrective ankle torque exists at all. A late action that arrives during that window does nothing.
2. **Time constant.** Balance is an inverted pendulum, $\tau = \sqrt{L/g} \approx 0.25$ s at this centre-of-mass height. 40 ms is about 16% of the fall time — survivable when you are simply holding still.
3. **Fixed point versus limit cycle.** Standing stabilises a point, so an error decays. Walking stabilises a repeating orbit, so delay shifts the *phase* — and phase error **compounds every step** instead of dying out.

The honest footnote, which the tool prints itself

Every row above is **one deterministic rollout** — which is exactly why it reproduces on your machine. The research run used 20 randomised episodes over 20 s and reported the fully-safe margins as **80 ms standing vs 20 ms walking**, a 4× gap rather than 6×. A single clean rollout is more forgiving than twenty disturbed ones.

The cliff location reproduces. The exact survival second does not — it also moves slightly between MuJoCo versions. Knowing which of your numbers is the robust one is the skill.

☒ Checkpoint: the two tolerances, the ratio, and all three reasons.

Part 5 — Attack 2: corrupt one sensor

The policy sees 45 numbers in six groups. This attack corrupts **one group at a time**, dosed in multiples of the noise that group already saw during training — so **x10** means ten times what the policy expects.

Predict first

Five candidate channels. Which one, corrupted, kills a **balance** policy first?

```
angvel  gravity  dofpos  dofvel  prev_action
```

Write your answer down before running anything.

The null control comes first

```
python exp3_sweep.py noise
```

About 76 seconds. The first thing it prints is not a result:

```
clean          max tilt  3.35 deg  survived
command zeroed max tilt  3.35 deg  survived
-> PASS, identical. The tool is honest.
```

The standing policy was trained with a fixed command and has never seen a different one. Pinning that channel to its training value **must** change nothing. It doesn't — to the hundredth of a degree.

Why this step exists. An earlier version of this experiment tried to use *noise* on that channel as the null control, and the outcome changed. The reason is worth knowing: the command channel's training standard deviation is $5.87e-06$, so normalisation divides by it and amplifies any noise you add by roughly **170,000x**, straight into the clip bound. A channel with no training variance cannot be dosed in "multiples of training noise" at all.

A result you cannot check is not a result. If the null control had failed, nothing below it would be worth reading.

The hunt

Expected:

sensor	x1	x5	x10	x20	
angvel	3/3	3/3	3/3	3/3	
gravity	3/3	3/3	0/3	0/3	<- breaks at x10
dofpos	3/3	3/3	2/3	0/3	<- breaks at x20
dofvel	3/3	3/3	3/3	2/3	

Gravity breaks first, at x10. Angular velocity survives everything tested.

Watch the failure:

```
python watch_policy.py --noise gravity 10 --headless
```

Expected:

```
exp2_stand.npz | gravity noise x10
fell at 1.92s
```

Why gravity

Projected gravity is the only absolute orientation reference in the entire observation. Every joint sensor is relative to the body — they tell the policy how it is folded, not which way is up. Angular velocity is a *rate*: to get an angle from it you integrate, and integration drifts.

Corrupt gravity and the policy loses which way is up. For a balance task that is unrecoverable.

A prediction that was wrong, on the record

The pre-registration for this experiment said **angular velocity** would be the severe one. It turned out to be among the most robust. It is written here because it was written down beforehand — a prediction only counts if you cannot revise it afterwards.

The two channels that are not sensors

`command` and `prev_action` had **no** training noise, so "multiples of training noise" is undefined for them and they do not belong in the table. `command` was your null control. `prev_action` is the policy's own last output fed back to it, so it is dosed in raw units instead:

```
prev_action noise sd 0.10    3/3 survived
prev_action noise sd 0.25    3/3 survived
prev_action noise sd 0.50    3/3 survived
prev_action noise sd 1.00    2/3 survived
```

Its own values run about ± 1 , so `sd 0.5` is already heavy corruption and it barely cares.

☒ **Checkpoint: name the channel that breaks, the dose it breaks at, and why that channel and not another.**

Part 6 — Attack 3: swap the simulator

This is the one that matters, and the one whose result is not what you expect.

Everything so far corrupted the *policy's input*. This attack changes the **world**: the same frozen weights, the same kinematic tree, **different collision geometry** — the second robot you built in Part 1. Both robots run their own copy of the policy and react to their own observations, exactly as two real machines would.

Predict first

Same policy, different simulator. **How long until it falls?**

Run it

```
python exp3_sweep.py sim2sim
```

Expected — standing:

t	leg RMS	base apart	heading apart	upright
1.0s	0.0095 rad	0.012 m	2.5 deg	both
10.0s	0.0050 rad	0.008 m	1.9 deg	both
20.0s	0.0036 rad	0.008 m	1.7 deg	both

Expected — walking:

t	leg RMS	base apart	heading apart	upright
1.0s	0.1415 rad	0.060 m	13.3 deg	both
10.0s	0.1719 rad	0.222 m	23.6 deg	both
20.0s	0.1186 rad	1.835 m	37.2 deg	both

Read it before you move on

Neither robot ever falls. Look at the `upright` column: `both`, the whole way down, in both tables.

The standing pair ends **8 mm apart** after twenty seconds. The walking pair ends **1.8 metres apart and facing 37° away from each other** — both still walking, neither in any trouble.

The finding

The sim2sim gap is not a fall. It is a drift.

Standing clamps joint divergence near **0.01 rad** and stays put — a fixed point, so the error has somewhere to decay to. Walking clamps near **0.10 rad**, ten times higher, and because walking stabilises a **limit cycle**, that bounded per-step difference in step geometry **integrates**. A thousand control steps later the two simulations are metres apart, and both robots are fine.

Feedback is not eliminating the physics gap. It is **bounding** it. What differs between the two policies is the level it clamps at.

Watch them peel apart

Create `watch_sim2sim.py`:

```
"""WATCH two simulators disagree -- Lab 3, Attack 3.
```

```
The same frozen policy runs in two different physics models at once. Both robots react to their own observations. You see model A solid, and model B drawn as an orange GHOST on top of it.
```

```
They start perfectly overlapped. Watch how long that lasts.
```

```
python watch_sim2sim.py                                # standing
python watch_sim2sim.py --policy walk3b.npz --vx 0.5    # walking -- the good one
```

```
python watch_sim2sim.py --policy walk3b.npz --vx 0.5 --openloop
python watch_sim2sim.py --headless # numbers only
```

WHAT TO LOOK FOR

standing the ghost stays welded to the robot for the whole 20 s
 walking the ghost peels away, and by 20 s it is metres away facing a different direction -- WITHOUT EITHER ROBOT FALLING
 --shots 0.5,2,10,20 save pictures instead of opening a window
 --openloop feedback off: B replays A's actions blindly. Both topple in about 2.5 s. This is what the physics gap does when no controller is fighting it.

KEYS: left-drag orbit - scroll zoom - Esc quit

"""

```
import argparse, os, sys, time
# --shots renders offscreen; WSL2 has no usable EGL, so osmesa (software GL)
# is the backend that works there. Must be set before the GL context exists.
os.environ.setdefault("MUJOCO_GL", "osmesa")
import numpy as np
import mujoco

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
from watch_policy import Robot, load_policy # noqa: E402

GHOST = np.array([0.95, 0.45, 0.10, 0.65], dtype=np.float32) # orange, see-through
FELL = 45.0
```

def yaw(q):

```
w, x, y, z = q
return np.degrees(np.arctan2(2*(w*z + x*y), 1 - 2*(y*y + z*z)))
```

def measure(ra, rb):

```
"""The three numbers that describe how far apart the two worlds are."""
return (np.sqrt(np.mean((ra.d.qpos[ra.qadr] - rb.d.qpos[rb.qadr]) ** 2)),
        float(np.linalg.norm(ra.d.qpos[:2] - rb.d.qpos[:2])),
        abs(yaw(ra.d.qpos[3:7]) - yaw(rb.d.qpos[3:7])))
```

def draw_ghost(scn, rb, reset=True):

```
"""Paint model B's body positions into model A's scene as spheres.
```

```
reset=True for the viewer, which gives us a SEPARATE user scene to own.
reset=False for the offscreen renderer, whose scene already holds model A
and the floor -- clearing it there would leave nothing but the ghost.
```

```
"""
```

```
if reset:
```

```
    scn.ngeom = 0
```

```
for i in range(1, rb.m.nbody):
```

```
    if scn.ngeom >= scn.maxgeom:
```

```
        break
```

```
    mujoco.mjv_initGeom(
```

```

        scn.geoms[scn.ngeom], mujoco.mjtGeom.mjGEOM_SPHERE,
        np.array([0.045, 0, 0]), rb.d.xpos[i].copy(),
        np.eye(3).flatten(), GHOST)
    scn.ngeom += 1

def run(a):
    A = a.xml_a or os.path.join(HERE, "model", "r1_walk_train.xml")
    B = a.xml_b or os.path.join(HERE, "model", "r1_walk_deploy_env.xml")
    act = load_policy(os.path.join(HERE, "policies", a.policy))
    ra, rb = Robot(A, a.vx), Robot(B, a.vx)

    print(f"{a.policy}" + (f" | vx {a.vx}" if a.vx else "") +
          (" | OPEN LOOP (B replays A's actions)" if a.openloop else " | closed loop"))
    print(f"  solid = {os.path.basename(A)}")
    print(f"  ghost = {os.path.basename(B)}")
    print("\n      t      leg RMS      base apart      heading apart      upright")
    print("    " + "-" * 58)

    total, next_report, dead = int(a.seconds / ra.dt), 0.0, None

    def one_step():
        nonlocal dead
        aA = act(ra.obs())
        ra.step(aA)
        # closed loop: B decides for itself.  open loop: B is fed A's actions
        # with its own policy switched off, so only the physics differs.
        rb.step(aA if a.openloop else act(rb.obs()))
        if dead is None and max(ra.tilt(), rb.tilt()) > FELL:
            dead = (ra.d.time)

    def report(t):
        rms, dist, dh = measure(ra, rb)
        up = "both" if max(ra.tilt(), rb.tilt()) < FELL else "FALLEN"
        print(f"      {t:5.1f}s      {rms:7.4f} rad      {dist:7.3f} m      "
              f"{dh:7.1f} deg      {up}")

    if a.headless:
        for s in range(total):
            one_step()
            t = (s + 1) * ra.dt
            if t >= next_report - 1e-9:
                report(t); next_report += a.every
            if dead is not None:
                # stop the clock at the fall.  Distances measured after a robot
                # is lying on the floor describe a corpse sliding, not a policy.
                report(t) if t < next_report - a.every else None
                break
        summarise(ra, rb, a, dead)
        return

    if a.shots:
        import imageio
        W, H = 900, 700

```

```

ra.m.vis.headlight.ambient[:] = [0.6, 0.6, 0.6]
ra.m.vis.headlight.diffuse[:] = [0.7, 0.7, 0.7]
ra.m.vis.global_.offwidth = max(ra.m.vis.global_.offwidth, W)
ra.m.vis.global_.offheight = max(ra.m.vis.global_.offheight, H)
cam = mujoco.MjvCamera()
mujoco.mjv_defaultCamera(cam)
cam.distance, cam.azimuth, cam.elevation = 3.5, 135, -8
ren = mujoco.Renderer(ra.m, height=H, width=W)
out = os.path.join(HERE, "lab_img")
os.makedirs(out, exist_ok=True)
want = [float(x) for x in a.shots.split(",")]
try:
    for s_i in range(total):
        one_step()
        t = (s_i + 1) * ra.dt
        if t >= next_report - 1e-9:
            report(t); next_report += a.every
        if want and t >= want[0] - 1e-9:
            cam.lookat[:] = [ra.d.qpos[0], ra.d.qpos[1], 0.45]
            ren.update_scene(ra.d, camera=cam)
            draw_ghost(ren.scene, rb, reset=False)
            name = "ghost_%s_%04.1fs.png" % (a.policy.split(".")[0], want[0])
            imageio.imwrite(os.path.join(out, name), ren.render())
            print("    wrote lab_img/%s" % name)
            want.pop(0)
        if dead is not None:
            break
finally:
    ren.close()
summarise(ra, rb, a, dead)
return

# `import mujoco.viewer` would rebind the name `mujoco` as a local of this
# function, which breaks the --shots branch above; import the submodule only.
from mujoco import viewer as mj_viewer
with mj_viewer.launch_passive(ra.m, ra.d) as v:
    v.cam.distance, v.cam.azimuth, v.cam.elevation = 3.5, 135, -8
    s = 0
    while v.is_running():
        t0 = time.time()
        if s < total:
            one_step(); s += 1
            t = s * ra.dt
            if t >= next_report - 1e-9:
                report(t); next_report += a.every
            if s == total:
                summarise(ra, rb, a, dead)
        draw_ghost(v.user_scn, rb)
        v.cam.lookat[:] = [ra.d.qpos[0], ra.d.qpos[1], 0.45]
        v.sync()
        lag = ra.dt - (time.time() - t0)
        if lag > 0:
            time.sleep(lag)

```



```

def summarise(ra, rb, a, dead):
    rms, dist, dh = measure(ra, rb)
    print()
    if dead is not None:
        print(f" A robot fell at {dead:.2f}s.")
        if a.openloop:
            print(" That is the point of open loop: with no controller reacting to")
            print(" the difference, contact errors compound and topple it in ~2.5 s.")
        else:
            print(f" NEITHER ROBOT FELL in {a.seconds:g}s -- and they ended {dist:.3f} m")
            print(f" apart, facing {dh:.0f} degrees away from each other.")
            print(" The sim2sim gap is not a fall. It is a drift.")
    print(f"\n final: leg RMS {rms:.4f} rad | {dist:.3f} m | {dh:.1f} deg\n")

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--policy", default="exp2_stand.npz")
    p.add_argument("--vx", type=float, default=0.0)
    p.add_argument("--seconds", type=float, default=20.0)
    p.add_argument("--every", type=float, default=2.0, help="seconds between rows")
    p.add_argument("--openloop", action="store_true",
                    help="B replays A's actions with its own policy OFF")
    p.add_argument("--xml-a", default=None)
    p.add_argument("--xml-b", default=None)
    p.add_argument("--headless", action="store_true")
    p.add_argument("--shots", default=None,
                    help="comma-separated seconds to save pictures at, "
                    "e.g. 0.5,2,10,20 -- no window needed")
    run(p.parse_args())

```

Two robots, same weights, different physics. The second is drawn as an **orange translucent ghost**. They start welded together, and you watch the ghost separate.

```
python watch_sim2sim.py --policy walk3b.npz --vx 0.5 --shots 0.5,2,10,20 --every 5
```

Expected:

t	leg RMS	base apart	heading apart	upright
0.0s	0.0006 rad	0.000 m	0.0 deg	both
wrote lab_img/ghost_walk3b_00.5s.png				
wrote lab_img/ghost_walk3b_02.0s.png				
5.0s	0.1270 rad	0.224 m	3.4 deg	both
10.0s	0.1719 rad	0.222 m	23.6 deg	both
wrote lab_img/ghost_walk3b_10.0s.png				
15.0s	0.0345 rad	0.888 m	29.2 deg	both
20.0s	0.1186 rad	1.835 m	37.2 deg	both
wrote lab_img/ghost_walk3b_20.0s.png				

NEITHER ROBOT FELL in 20s -- and they ended 1.835 m apart, facing 37 degrees away from each other. The sim2sim gap is not a fall. It is a drift.

Open the four PNGs in order. At 0.5 s the ghost is inside the robot. At 20 s it is most of the way across the picture, **still upright and still walking**. That single image is the finding of this lab.

`--shots` needs no window, so it works under WSL2. If your machine does open MuJoCo windows, run it live instead — no flags, and the ghost separates in front of you:

```
python watch_sim2sim.py --policy walk3b.npz --vx 0.5
```

The control — turn the feedback off

If feedback is what bounds the gap, then removing it should unbound it. `--openloop` switches the second robot's policy off entirely and replays the first robot's actions into it blindly:

```
python watch_sim2sim.py --openloop --headless --every 1
```

Expected:

0.0s	0.0006 rad	0.000 m	0.0 deg	both
1.0s	0.0105 rad	0.019 m	2.0 deg	both
2.0s	0.0455 rad	0.100 m	1.4 deg	both

A robot fell at 2.68s.

Same two simulators. Same policy. The only change is whether anything is reacting to the difference — and the robot that was upright at twenty seconds is now down in under three.

That is the proof. The physics gap was always there; in closed loop it was being corrected away 50 times a second. Open the loop and contact errors compound instead of being cancelled.

An honest limitation, before anyone asks

Neither simulator is ground truth. **This measures disagreement between two models, not error against reality.** Only hardware settles that. Say it before you are asked — a measurement presented without its limitation invites the question that undermines it.

☑ Checkpoint: state what drifted, what did not fall, and what the open-loop run proves.

Part 7 — Synthesis

Three attacks, three different lessons:

Attack	Standing	Walking	Would a real robot see this?
Control delay	120 ms	20 ms	Yes — every real system has latency
Corrupt gravity	breaks at $\times 10$	—	Yes — IMUs drift and get knocked
Swap the simulator	8 mm drift	1.8 m drift	Yes, as drift — not as a fall

Now put them together. **The policy survived a ten-times corrupted joint sensor and a sixth of a second of delay. It did not fall when the entire world model changed.** So when a policy trained in simulation misbehaves on real hardware, the sensing pipeline is the wrong first suspect.

Where the sim2sim gap actually lives

Not in sensing. This project spent weeks assuming otherwise — the walking policy transferred badly and domain randomisation was the obvious fix. It was not: the policy walks 20 s on the exact deployment model. The gap was in the **real-time control harness** — timing, not perception.

Part 4 is the clue you were given first. Delay is what walking cannot tolerate, and a real-time harness is a machine for producing delay.

Change something

`exp3_sweep.py` has three **EDIT ZONE** blocks. Pick one and re-run the section:

- **Zone 1** — the policies that get attacked. Add your own `my_run.npz` from Lab 2 and see whether a half-trained policy has the same delay budget as a finished one.
- **Zone 2** — the delay ladder. The cliff for walking sits between 20 and 40 ms. Put three rows in there and find it more precisely.
- **Zone 3** — the noise doses. Gravity broke somewhere between $\times 5$ and $\times 10$. Where exactly?

Any of those is a real measurement that is not in this manual.

What this lab was really teaching

Every part had a control:

- Part 2 established that both policies work **before** anything was attacked.
- Part 5 ran a null control **before** reporting which sensor mattered.
- Part 6 ran an open-loop control to prove that feedback, not luck, was bounding the drift.

A measurement without a control is an anecdote. That is the transferable skill, and it is worth more than any single number in the tables.

Troubleshooting

Symptom	Cause & fix
<code>ModuleNotFoundError: mujoco</code>	<code>conda activate rllab</code> — needed in every new terminal
<code>cannot find ... R1_C++.xml</code>	your Lab 1 clone is gone. Redo Lab 1 Part 2, or pass <code>--src</code>
<code>resource not found ... pelvis_link.STL</code>	<code>model/assets</code> missing — rerun the <code>cp -r</code> in Part 0
<code>FileNotFoundError: walk3b.npz</code>	the download did not land in <code>policies/</code>
<code>gladLoadGL error</code> on the viewers	no usable window. Add <code>--headless</code> , or <code>--shots</code> for pictures
The ghost robot is invisible	it starts welded to the first one — wait a few seconds
Sweep seems stuck	<code>all</code> takes about 90 seconds and prints nothing between sections
Numbers differ from this manual	tell the instructor — the policies are frozen and the physics is deterministic, so on this MuJoCo version they should not

What to hand in

1. Your two predictions from Parts 4 and 5, written **before** running.
2. The output of `build_deploy_model.py`, and one sentence on why change #5 was necessary.
3. The latency table, with the two tolerance numbers and the ratio.
4. One paragraph: **why does walking tolerate so much less delay?** Give all three reasons.
5. The sensor that broke first, its dose, and why that channel.
6. **What the null control was, and what it would have meant if it had failed.**
7. The two 20-second drift figures, and one sentence on why they differ by 200×.
8. One sentence: **what does the open-loop run prove?**
9. One edit-zone change of your own, and the number it produced.

Appendix — Instructor Notes

- **Parts 4 and 5 are worthless without the predictions.** Make them write the numbers down. The 6× delay ratio and "not angular velocity" both land far harder against a committed guess.
- **Part 6 is the lab.** Students arrive expecting a fall and there isn't one. Let the confusion sit for a moment before explaining the drift — it is the most useful silence in the workshop.
- **Part 1 is new in this version and worth the ten minutes.** Students who *build* the second robot from Unitree's own file never ask whether the sim2sim gap was rigged. The script prints its six changes and the compiled mass matches Labs 1 and 2 to the gram.
- **The null control is the most transferable five minutes here.** It is also a real bug story: the first version of it was itself broken, by a 170,000× amplification nobody predicted.
- **Say the limitation out loud.** Neither simulator is ground truth. Students who repeat this claim without it will be caught out by the first person who asks.
- **--shots exists because WSL2 has no working MuJoCo window** on many machines. The ghost picture at 20 s is the single most useful image in the workshop; do not let a graphics failure cost it.
- **Timing:** Part 0–1 about 15 min, Part 2–3 ten, Parts 4–6 about 15 each including the 90 s of compute, Part 7 five. The sweeps run while you talk.
- Everything is CPU-only, deterministic, and needs nothing beyond Lab 1's install.